Ejercicios con los TAD Lista, Pila y Cola

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Escoger entre lista, pila y cola a partir del orden en que un problema necesita entrar y sacar sus datos.
- Resolver problemas clásicos usando solo el contrato: balance de paréntesis, evaluación postfija, eliminación circular, intersección de listas ordenadas.
- Trazar a mano el estado de la estructura en cada paso y detectar dónde una solución viola una precondición.
- Construir un TAD sobre otro: una cola hecha con dos pilas.
- Decir qué cuesta cada solución en tiempo y en espacio, a partir de los costos de las operaciones del contrato.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: diseñar e implementar soluciones a problemas pequeños con estructuras de datos simples.
- OA2: escoger la estructura por el costo de las operaciones que el problema pide.
- OA3: argumentar con precisión, aquí en forma de trazas y precondiciones.

Referencia bibliográfica

R. Thareja, *Data Structures Using C*, 2018, capítulo 7 (aplicaciones de pilas: balance de paréntesis, notación postfija) y capítulo 8 (aplicaciones de colas). N. Kalicharan, *Data Structures in C*, 2008, capítulo 4.

Contenido I

1 Los tres contratos en una página

2 Con una pila

3 Con una cola

4 Con una lista

5 Un TAD sobre otro

Contenido I

1 Los tres contratos en una página

2 Con una pila

3 Con una cola

4 Con una lista

5 Un TAD sobre otro

Lo que cada uno permite I

	Lista	Pila	Cola
entra por	cualquier p	el tope	el final
sale por	cualquier p	el tope	el frente
se lee	cualquier p	solo el tope	solo el frente
operaciones	`insertar`, `eliminar`, `obtener`, `asignar`, `agregar`	`apilar`, `desapilar`, `tope`	`encolar`, `desencolar`, `frente`
precondición	$0 \leq p < n$	$n > 0$	$n > 0$

Lo que cada uno permite II

Cómo se ataca un ejercicio con un TAD

1. Decir qué se guarda y en qué orden tiene que salir. Eso escoge el TAD.
2. Escribir, en palabras, qué operación se hace en cada paso.
3. Vigilar las precondiciones: antes de `tope` o `frente`, ¿quién asegura que no está vacía?
4. Trazar con un caso pequeño antes de dar la solución por buena.
5. Decir qué cuesta: cuántas operaciones del contrato hace en función de n, y cuánto espacio pide aparte de la estructura. Las operaciones de pila y cola cuestan $O(1)$; en la lista, `obtener` y `agregar` $O(1)$, `insertar` y `eliminar` en medio $O(n)$.

Contenido I

1 Los tres contratos en una página

2 Con una pila

3 Con una cola

4 Con una lista

5 Un TAD sobre otro

Paréntesis balanceados I

Problema

Dada una cadena con `(`, `)`, `[` y `]`, decidir si está bien balanceada: cada apertura tiene su cierre, del mismo tipo, y en el orden correcto. `([])()` sí; `([)]` no.

Pregunta

Al leer `([)]` de izquierda a derecha, ¿en qué carácter se sabe que está mal? ¿Qué había que recordar para saberlo?

- Al llegar al `)` hay que saber cuál fue la **última** apertura sin cerrar. No la primera: la última.
- Recordar lo último que llegó y olvidarlo cuando se cierra es la disciplina LIFO. Thareja, capítulo 7.

Paréntesis balanceados II

```
// dice si c cierra lo que a abre
bool cierra(Elemento a, Elemento c) {
  return (a == '(' && c == ')') || (a == '[' && c == ']');
}
```

Paréntesis balanceados III

```
bool balanceado(const char *s) {
  Pila p;
  bool ok = true;
  int i = 0;
  while (ok && s[i] != '\0') {
    if (s[i] == '(' || s[i] == '[') {
      p.apilar(s[i]);
    } else if (p.vacia()) {
      ok = false;
    } else if (cierra(p.tope(), s[i])) {
      p.desapilar();
    } else {
      ok = false;
    }
    i = i + 1;
  }
  return ok && p.vacia();
}
```

La traza de balanceado I

`([])()`

i	$s[i]$	acción	pila (tope a la derecha)
0	(	apilar	$\langle (\rangle$
1	[	apilar	$\langle (, [\rangle$
2	]	cierra [: desapilar	$\langle (\rangle$
3	)	cierra (: desapilar	$\langle \rangle$
4	(	apilar	$\langle (\rangle$
5	)	cierra (: desapilar	$\langle \rangle$
fin		ok y vacía: **sí**	

La traza de balanceado II

`([)]`

i	$s[i]$	acción	pila
0	`(`	apilar	$\langle ($
1	`[`	apilar	$\langle (, [$
2	`)`	el tope es `[`, no cierra: `ok = false`	$\langle (, [$
fin		**no**	

La traza de balanceado III

Pregunta: dos casos que engañan

¿Qué responde con `((`? ¿Y con `)(`? ¿Qué parte del código decide cada uno?

- `((`: el ciclo termina con `ok` en verdadero y la pila con dos elementos. Lo rechaza el `p.vacia()` del `return`. Sin esa mitad, toda cadena de puras aperturas pasaría.
- `)(`: en $i = 0$ la pila está vacía y llega un cierre. Lo rechaza la rama `p.vacia()` del ciclo, que además impide llamar a `tope` sobre una pila vacía. Esa rama es la precondición de `tope`, vigilada desde el área del programador.
- La cadena vacía es balanceada: no entra al ciclo y la pila está vacía.

La traza de balanceado IV

- Costo, con n la longitud de la cadena: $\Theta(n)$ en tiempo, una vuelta por carácter con operaciones de $O(1)$; $O(n)$ de espacio para la pila, que en el peor caso (una cadena de puras aperturas) guarda los n caracteres.

Evaluar una expresión postfija I

Problema

En notación postfija el operador va después de sus dos operandos: `3 4 + 2 *` es $(3+4)\cdot 2 = 14$. No hay paréntesis ni precedencia. Evaluar una expresión dada como cadena.

Idea

Se lee de izquierda a derecha. Un número se guarda; un operador toma los **dos últimos** números guardados, opera y guarda el resultado. Al final queda un solo número.

- "Los dos últimos" otra vez es LIFO. Las calculadoras HP y la máquina virtual de Java evalúan así.
- Cuidado con el orden: para `5 3 -` el segundo que sale de la pila es el primer operando.

Evaluar una expresión postfija II

```
int aplicar(char op, int a, int b) {
  int r = 0;
  if (op == '+') {
    r = a + b;
  } else if (op == '-') {
    r = a - b;
  } else if (op == '*') {
    r = a * b;
  } else {
    r = a / b;
  }
  return r;
}
```

Evaluar una expresión postfija III

```
// s trae digitos y operadores separados por espacios; exige que sea
↪  valida
int evaluar(const char *s) {
  Pila p;
  int i = 0;
  while (s[i] != '\0') {
    if (s[i] >= '0' && s[i] <= '9') {
      p.apilar(s[i] - '0');
    } else if (s[i] != ' ') {
      int b = p.tope();
      p.desapilar();
      int a = p.tope();
      p.desapilar();
      p.apilar(aplicar(s[i], a, b));
    }
    i = i + 1;
  }
  return p.tope();
}
```

Evaluar una expresión postfija IV

- Números de un solo dígito, para que toda la atención esté en la pila. Leer números de varias cifras es un ciclo más, y queda propuesto.

La traza de evaluar con `3 4 + 2 * l`

se lee	acción	pila (tope a la derecha)	
3	apilar 3	$\langle 3 \rangle$	
4	apilar 4	$\langle 3, 4 \rangle$	
+	$b = 4$, $a = 3$, apilar $3 + 4$	$\langle 7 \rangle$	
2	apilar 2	$\langle 7, 2 \rangle$	
*	$b = 2$, $a = 7$, apilar $7 \cdot 2$	$\langle 14 \rangle$	
fin	`tope()`		**14**

Pregunta

¿Qué da `5 3 -`? ¿Y si en el código se intercambiaran las dos líneas que leen b y a?

La traza de evaluar con `3 4 + 2 *` II

- $5 - 3 = 2$: el 3 es el tope y sale primero, por eso se guarda en b. Con las líneas intercambiadas daría -2: el programa compila, no viola ninguna precondición y responde mal. La pila no sabe de aritmética; el orden lo pone el programador.
- Salidas reales: `8 2 / 3 -` da 1 y `2 3 4 * +` da 14.
- Costo, con n la longitud de la cadena: $\Theta(n)$ en tiempo, cada carácter dispara a lo sumo tres operaciones de $O(1)$; $O(n)$ de espacio para la pila de operandos, que crece cuando los números vienen antes que sus operadores (`2 3 4 * +` llega a tener tres).

Contenido I

1 Los tres contratos en una página

2 Con una pila

3 Con una cola

4 Con una lista

5 Un TAD sobre otro

Calentamiento en parejas: imprimir una cola sin destruirla I

Enunciado

Escriba `imprimir`(c) para una cola, del frente al final, de modo que al terminar c quede exactamente como estaba. Con $\langle 7, 2, 9, 4 \rangle$ imprime `7 2 9 4`.

Pistas

- En la pila hizo falta una auxiliar porque lo que se devuelve entra por el mismo extremo por el que salió. En la cola, ¿por dónde entra lo que se devuelve?
- `particionar` ya recorrió una cola completa. ¿Cómo supo cuándo parar?

Calentamiento en parejas: imprimir una cola sin destruirla II

Una solución

```
void imprimir(Cola &c) {
  int i = 0;
  int n = c.tamano();
  while (i < n) {
    Elemento x = c.frente();
    printf(" %d", x);
    c.desencolar();
    c.encolar(x);
    i = i + 1;
  }
  printf("\n");
}
```

Calentamiento en parejas: imprimir una cola sin destruirla III

- Sin auxiliar: al dar la vuelta completa cada elemento vuelve a su lugar. Una pasada, no dos: la disciplina FIFO se paga en otras cosas, no en esta.
- El tamaño se lee una vez, antes del ciclo. Con `!c.vacia()` el ciclo no terminaría nunca.

La papa caliente I

Problema

n personas en círculo, numeradas de 1 a n. Se cuenta desde la primera y la k-ésima sale del juego; se sigue contando desde la siguiente. ¿En qué orden salen y quién queda de última?

Pregunta

¿Qué operación es "pasar la papa a la siguiente persona" con una cola? ¿Y "sale del juego"?

- Pasar la papa es sacar del frente y volver a encolar: la persona va al final del círculo.
- Salir del juego es desencolar sin volver a encolar.
- El círculo es una cola que da vueltas. Thareja, capítulo 8, presenta la versión circular de la cola justo para esto.

La papa caliente II

```
// devuelve quien queda de ultimo; imprime el orden en que salen
int ultimo(int n, int k) {
  Cola c;
  int i = 1;
  while (i <= n) { c.encolar(i); i = i + 1; }
  while (c.tamano() > 1) {
    int j = 1;
    while (j < k) {
      Elemento x = c.frente();
      c.desencolar();
      c.encolar(x);
      j = j + 1;
    }
    printf("sale %d\n", c.frente());
    c.desencolar();
  }
  return c.frente();
}
```

La traza con $n = 5$ y $k = 3$ I

ronda	cola antes (frente a la izquierda)	pasan $k - 1 = 2$, sale
1	$\langle 1, 2, 3, 4, 5 \rangle$	$\langle 3, 4, 5, 1, 2 \rangle$: sale **3**
2	$\langle 4, 5, 1, 2 \rangle$	$\langle 1, 2, 4, 5 \rangle$: sale **1**
3	$\langle 2, 4, 5 \rangle$	$\langle 5, 2, 4 \rangle$: sale **5**
4	$\langle 2, 4 \rangle$	$\langle 2, 4 \rangle$: sale **2**
fin	$\langle 4 \rangle$	queda **4**

- En la ronda 4 la cola tiene dos personas y hay que pasar la papa dos veces: 2 va al final, luego 4 va al final, y el frente vuelve a ser 2. Sin trazarlo, es fácil creer que sale el 4.
- El ciclo externo termina cuando queda una persona; con $n = 1$ no entra y devuelve la única. Con $n = 0$ el `frente` final violaría la precondición: el enunciado exige $n \geq 1$.
- Salida real del programa: `sale 3, sale 1, sale 5, sale 2, queda 4`.

La traza con $n = 5$ y $k = 3$ II

- Costo: cada ronda hace $k - 1$ pases de $O(1)$ y una salida, y hay $n - 1$ rondas: $\Theta(nk)$ en tiempo; $\Theta(n)$ de espacio, la cola con las n personas. Con k grande frente a n se pasa la papa muchas vueltas completas; k mód tamaño actual las ahorra, y queda propuesto.

Ejercicio en parejas: invertir una cola I

Enunciado

Escriba `invertir`(c), que deja la cola con sus elementos en orden inverso: $\langle 7, 2, 9, 4 \rangle$ pasa a $\langle 4, 9, 2, 7 \rangle$. Puede usar una pila auxiliar.

Pistas

- Dar la vuelta completa no invierte nada: cada elemento vuelve a su lugar. Hace falta algo que invierta.
- ¿Qué estructura devuelve las cosas en orden contrario al que entraron?

Ejercicio en parejas: invertir una cola II

Una solución

```
void invertir(Cola &c) {
  Pila p;
  while (!c.vacia()) {
    p.apilar(c.frente());
    c.desencolar();
  }
  while (!p.vacia()) {
    c.encolar(p.tope());
    p.desapilar();
  }
}
```

- Dos pasadas por n elementos. La cola entra a la pila en su orden y sale al revés: eso es todo lo que la pila hace, y es justo lo que faltaba.

Ejercicio en parejas: invertir una cola III

- Con dos colas no se puede: cualquier combinación de desencolar y encolar conserva el orden relativo. Vale la pena convencerse de eso antes de intentarlo.
- Costo: $\Theta(n)$ en tiempo, $2n$ operaciones de $O(1)$; $\Theta(n)$ de espacio, la pila auxiliar recibe los n elementos.

Contenido I

1 Los tres contratos en una página

2 Con una pila

3 Con una cola

4 Con una lista

5 Un TAD sobre otro

Quitar repetidos de una lista ordenada I

Problema

l viene ordenada de menor a mayor y puede traer valores repetidos. Dejar una sola copia de cada valor, sin crear otra lista: $\langle 1, 1, 2, 3, 3, 3, 5 \rangle$ queda $\langle 1, 2, 3, 5 \rangle$.

```
// l viene ordenada; deja una sola copia de cada valor
void quitarRepetidos(Lista &l) {
  int p = 0;
  while (p < l.tamano() - 1) {
    if (l.obtener(p) == l.obtener(p + 1)) {
      l.eliminar(p + 1);
    } else {
      p = p + 1;
    }
  }
}
```

Quitar repetidos de una lista ordenada II

Traza

p	compara	acción	lista
0	$1 = 1$	eliminar(1)	$\langle 1, 2, 3, 3, 3, 5 \rangle$
0	$1 \neq 2$	$p = 1$	
1	$2 \neq 3$	$p = 2$	
2	$3 = 3$	eliminar(3)	$\langle 1, 2, 3, 3, 5 \rangle$
2	$3 = 3$	eliminar(3)	$\langle 1, 2, 3, 5 \rangle$
2	$3 \neq 5$	$p = 3$	
3	$3 < 4 - 1$ no	fin	$\langle 1, 2, 3, 5 \rangle$

Pregunta

¿Por qué p no avanza cuando elimina? ¿Qué pasaría con $\langle 3, 3, 3 \rangle$ si avanzara siempre?

Quitar repetidos de una lista ordenada III

- Al eliminar la posición $p + 1$, el siguiente elemento se corre a esa misma posición: hay que volver a compararlo con e_p. Si p avanzara, $\langle 3, 3, 3 \rangle$ quedaría en $\langle 3, 3 \rangle$.
- La condición $p < n - 1$ se reevalúa con el tamaño nuevo en cada vuelta; por eso el ciclo termina aunque la lista se acorte.

Costo

Cada `eliminar`$(p + 1)$ corre lo que sigue: $O(n)$ con la lista sobre arreglo. Si hay d repetidos, el tiempo es $O(n + d \cdot n)$: $\Theta(n)$ cuando no hay repetidos y $\Theta(n^2)$ en el peor caso (todos iguales). El espacio aparte de la lista es $\Theta(1)$, solo p.

- La otra opción es construir una lista nueva con `agregar` cada valor que difiera del anterior: $\Theta(n)$ en tiempo y $\Theta(n)$ de espacio. Se cambia espacio por tiempo, y el enunciado, al pedir "sin crear otra lista", eligió la versión en sitio.

Ejercicio en parejas: intersección de listas ordenadas I

Enunciado

a y b vienen ordenadas y sin repetidos. Escriba `interseccion`(a, b), que devuelve una lista nueva con los valores que están en ambas, ordenada: $\langle 1, 3, 4, 7, 9 \rangle$ y $\langle 2, 3, 7, 8, 9 \rangle$ dan $\langle 3, 7, 9 \rangle$.

Pistas

- Es la estructura de `mezclar`: dos posiciones que avanzan. Cambia qué se hace en cada comparación.
- Hay tres casos, no dos: iguales, menor en a, menor en b. ¿En cuál avanzan las dos posiciones?

Ejercicio en parejas: intersección de listas ordenadas II

Una solución

```
Lista interseccion(Lista &a, Lista &b) {
  Lista r;
  int i = 0;
  int j = 0;
  while (i < a.tamano() && j < b.tamano()) {
    if (a.obtener(i) == b.obtener(j)) {
      r.agregar(a.obtener(i));
      i = i + 1;
      j = j + 1;
    } else if (a.obtener(i) < b.obtener(j)) {
      i = i + 1;
    } else {
      j = j + 1;
    }
  }
  return r;
}
```

Ejercicio en parejas: intersección de listas ordenadas III

- El menor de los dos frentes no puede estar en la otra lista más adelante, porque están ordenadas: se descarta sin mirar más.
- No hay "copiar el resto": lo que sobra en una lista no tiene pareja. Costo: a lo sumo $\texttt{tamano}(a) + \texttt{tamano}(b)$ comparaciones.
- Costo: $\Theta(\texttt{tamano}(a) + \texttt{tamano}(b))$ en tiempo, porque cada vuelta avanza al menos una posición y todas las operaciones son $O(1)$; espacio $O(\text{mín}(\texttt{tamano}(a), \texttt{tamano}(b)))$, la lista nueva, que no puede tener más elementos que la más corta.

Contenido I

1 Los tres contratos en una página

2 Con una pila

3 Con una cola

4 Con una lista

5 Un TAD sobre otro

Una cola hecha con dos pilas I

Problema

Construir el contrato de Cola usando **solo** el contrato de Pila: `encolar`, `desencolar`, `frente` y `vacia`.

Idea

Una pila invierte el orden. Dos pilas lo invierten dos veces, y dos inversiones dejan el orden original. Lo que entra se apila en `entrada`; cuando hace falta el frente y `salida` está vacía, se pasa todo de `entrada` a `salida`: el primero que entró queda en el tope.

- Es la misma idea de `invertir`, pero sin devolver los elementos a la cola: se quedan en la segunda pila, ya en el orden correcto para salir.

Una cola hecha con dos pilas II

```
class ColaDP {
private:
  Pila entrada;
  Pila salida;
  // pasa todo lo que hay en entrada a salida, invirtiendo el orden
  void trasvasar() {
    while (!entrada.vacia()) {
      salida.apilar(entrada.tope());
      entrada.desapilar();
    }
  }
```

- Lo privado: dos pilas y el trasvase. Sigue la parte pública de la misma clase; `frente` y `desencolar` exigen cola no vacía.

Una cola hecha con dos pilas III

```
public:
  void encolar(Elemento e) {
    entrada.apilar(e);
  }
  Elemento frente() {
    if (salida.vacia()) {
      trasvasar();
    }
    return salida.tope();
  }
  void desencolar() {
    if (salida.vacia()) {
      trasvasar();
    }
    salida.desapilar();
  }
  bool vacia() { return entrada.vacia() && salida.vacia(); }
};
```

La traza de la cola con dos pilas I

operación	`entrada` (tope a la derecha)	`salida` (tope a la derecha)	responde
encolar 7, 2, 9	$\langle 7, 2, 9 \rangle$	$\langle\,\rangle$	
frente	$\langle\,\rangle$	$\langle 9, 2, 7 \rangle$	7
desencolar	$\langle\,\rangle$	$\langle 9, 2 \rangle$	
encolar 4	$\langle 4 \rangle$	$\langle 9, 2 \rangle$	
frente	$\langle 4 \rangle$	$\langle 9, 2 \rangle$	2
desencolar, desencolar	$\langle 4 \rangle$	$\langle\,\rangle$	
frente	$\langle\,\rangle$	$\langle 4 \rangle$	4

- El 4 entró después que el 9 y sale después: FIFO cumplido, aunque por dentro solo haya pilas.
- Trasvasar cuesta tantos `apilar` como elementos haya, pero cada elemento se trasvasa **una sola vez** en su vida. Repartido entre todas las operaciones, cada una cuesta constante. Ese argumento tiene nombre, costo amortizado, y vuelve con las tablas hash.

La traza de la cola con dos pilas II

- Salida real: `frente: 7, frente: 2, frente: 4, vacia: 1`.

Costos

`encolar`: $O(1)$. `frente` y `desencolar`: $O(n)$ en la llamada que trasvasa y $O(1)$ en las demás; como cada elemento se trasvasa una sola vez, n operaciones cualesquiera cuestan $O(n)$ en total: $O(1)$ amortizado por operación. Espacio: $\Theta(n)$, repartido entre las dos pilas.

Lo que costó cada solución I

Función	Tiempo	Espacio aparte de la estructura
`balanceado`	$\Theta(n)$	$O(n)$, pila de aperturas
`evaluar`	$\Theta(n)$	$O(n)$, pila de operandos
`ultimo`(n, k)	$\Theta(nk)$	$\Theta(n)$, la cola
`invertir`	$\Theta(n)$	$\Theta(n)$, pila auxiliar
`quitarRepetidos`	$O(n^2)$; $\Theta(n)$ sin repetidos	$\Theta(1)$
`interseccion`	$\Theta(n_a + n_b)$	$O(\text{mín}(n_a, n_b))$, lista nueva
`ColaDP`	$O(1)$ amortizado por operación	$\Theta(n)$, las dos pilas

- En pila y cola el costo de la solución es el número de operaciones, porque cada una vale $O(1)$. En la lista hay que mirar cuál operación: `obtener` y `agregar` valen $O(1)$, `eliminar` en medio $O(n)$, y eso vuelve cuadrático a `quitarRepetidos`.
- El espacio de una función es lo que pide aparte: una pila auxiliar, una lista nueva. Devolver una lista cuesta Θ de su tamaño aunque el tiempo sea el mismo.

Para el juez: UVa 673 — Parentheses Balance I

`https://vjudge.net/problem/UVA-673`
Envío: `https://vjudge.net/problem/UVA-673`

Qué pide

Cadenas hechas solo de `()` y `[]`. Una cadena es correcta si es vacía, si es la concatenación de dos correctas, o si es `(A)` o `[A]` con `A` correcta. Es `balanceado` tal cual: cada apertura se apila y cada cierre debe casar con el tope.

- **Entrada.** Una línea con n, y luego n líneas, una cadena por línea. Una línea puede venir vacía.
- **Salida.** Por cada cadena, `Yes` o `No` en su propia línea.
- **Límites.** Cada cadena tiene a lo sumo 128 caracteres: una pila de 128 alcanza. Cuesta $\Theta(L)$ por línea, con $O(L)$ de pila.

Para el juez: UVa 673 — Parentheses Balance II

Sample Input

```
3
([])
(([()]))
([()[]()])()
```

Sample Output

```
Yes
No
Yes
```

La trampa: la línea vacía

Es correcta y vale `Yes`. `scanf("%s")` se la salta y toma la siguiente línea en su lugar: todas las respuestas se corren una posición y al final falta una. Lea n con `scanf`, consuma el resto de esa línea con un `fgets`, y de ahí en adelante un `fgets` por cadena, quitando el `\n` final antes de recorrerla. Con 2, una línea vacía y `()`, la salida es `Yes` y luego `No`.

Para el juez: UVa 10935 I

Throwing cards away I.
`https://vjudge.net/problem/UVA-10935`
Envío: `https://vjudge.net/problem/UVA-10935`

Qué pide

Un mazo con las cartas $1, \ldots, n$ en orden, la 1 arriba. Mientras queden al menos dos: se descarta la de arriba y la que queda arriba pasa al fondo. Reportar las descartadas, en orden, y la que queda.

- **Entrada.** Una línea por caso con $n \leq 50$. La última línea trae 0 y no se procesa.
- **Salida.** Dos líneas por caso: `Discarded cards: 1, 3, 5, 7, 4, 2`, con coma y espacio entre cartas y sin espacio al final, y `Remaining card: 6`.

Para el juez: UVa 10935 II

Con una cola

Encole $1, \ldots, n$. Mientras el tamaño sea mayor que 1: desencole (descartada); desencole y encole (pasa al fondo). Es la papa caliente con otra regla: $n-1$ descartes y $n-1$ pases, cada uno $O(1)$: $\Theta(n)$ en tiempo y $\Theta(n)$ la cola.

Traza con $n = 7$

Cola (frente a la izquierda)	Descartada	Pasa al fondo
$\langle 1,2,3,4,5,6,7 \rangle$	1	2
$\langle 3,4,5,6,7,2 \rangle$	3	4
$\langle 5,6,7,2,4 \rangle$	5	6
$\langle 7,2,4,6 \rangle$	7	2
$\langle 4,6,2 \rangle$	4	6
$\langle 2,6 \rangle$	2	6
$\langle 6 \rangle$	queda la 6	

Para el juez: UVa 10935 III

Sample Input

```
7
6
1
0
```

Sample Output

```
Discarded cards: 1, 3, 5, 7, 4, 2
Remaining card: 6
Discarded cards: 1, 3, 5, 2, 6
Remaining card: 4
Discarded cards:
Remaining card: 1
```

Las trampas

Con $n = 1$ no se descarta nada: la primera línea es `Discarded cards:` y termina ahí, sin espacio. La coma y el espacio van antes de cada carta menos la primera, no después de cada una. Se lee hasta encontrar el 0.

Para el juez: UVa 514 — Rails I

`https://vjudge.net/problem/UVA-514`
Envío: `https://vjudge.net/problem/UVA-514`

Qué pide

Los vagones $1, \ldots, N$ llegan en ese orden por la vía A a una estación de una sola vía sin salida y siguen por la vía B; el que entró no vuelve a A y el que salió no regresa. ¿Pueden salir en el orden $a_1, \ldots, a_N$? La estación es una pila.

- **Entrada.** Bloques: una línea con $N \leq 1000$, varias líneas con una permutación de $1, \ldots, N$ cada una, y una línea `0` que cierra el bloque. Un `0` donde iría N termina la entrada.
- **Salida.** `Yes` o `No` por cada permutación, y una línea en blanco al terminar cada bloque. Nada por el `0` final.

Para el juez: UVa 514 — Rails II

Con una pila

Lleve `siguiente`, el próximo vagón que puede entrar, desde 1. Para cada a_i: mientras el tope no sea a_i y queden vagones, apile `siguiente` y avance; si el tope es a_i, desapile; si no, la respuesta es `No`. Cada vagón se apila y desapila a lo sumo una vez: $\Theta(N)$ por permutación con una pila de hasta N.

Traza con `5 4 1 2 3`

Pedido	Entran	Pila (tope a la derecha)	Sale
5	1, 2, 3, 4, 5	$\langle 1,2,3,4,5 \rangle$	5
4		$\langle 1,2,3,4 \rangle$	4
1	no quedan	$\langle 1,2,3 \rangle$, tope 3	`No`

Con `1 2 3 4 5` cada vagón entra y sale de inmediato: `Yes`.

Para el juez: UVa 514 — Rails III

Sample Input

```
5
1 2 3 4 5
5 4 1 2 3
0
6
6 5 4 3 2 1
0
0
```

Sample Output

```
Yes
No

Yes
```

Las trampas

La línea en blanco va después de cada bloque, también del último con datos. Cuando la respuesta ya es `No` a mitad de una permutación, hay que seguir leyendo los N números de esa línea antes de pasar a la siguiente. Vacíe la pila antes de cada permutación.

Propuestos para la casa I

1. **Llaves también.** Extienda `balanceado` para que acepte { y }. ¿Cuántas líneas cambian?
2. **Varias cifras.** Haga que `evaluar` lea números de más de un dígito: `12 2 / 3 -` debe dar 3.
3. **Postfija con error.** Haga que `evaluar` devuelva un indicador de error cuando la expresión esté mal formada: un operador sin dos operandos, o más de un número al final. Escriba primero qué precondición de la pila se violaría en cada caso.
4. **Palíndromo.** Con una pila y una cola, decida si una palabra se lee igual al derecho y al revés. Explique por qué hacen falta las dos.
5. **Unión ordenada.** Con la estructura de `interseccion`, escriba `union` de dos listas ordenadas sin repetidos: cada valor una sola vez.

Propuestos para la casa II

6. **Una pila con dos colas.** Es posible, pero una de las operaciones cuesta n. Escríbala y diga cuál.

Resumen I

- Pila cuando lo que importa es **lo último** que llegó: paréntesis abiertos, operandos pendientes.
- Cola cuando lo que importa es **el orden de llegada**: la papa caliente, los turnos.
- Lista cuando hace falta **la posición**: quitar repetidos, intersecar por dos posiciones.
- Las precondiciones se vigilan desde el área del programador: `p.vacia()` antes de `tope`, $n \geq 1$ antes de `frente`, $p < n - 1$ antes de `obtener(p + 1)`.
- Un TAD se puede construir con otro: la cola con dos pilas cumple FIFO y cobra constante amortizado.

Resumen II

- Todo lo de hoy es lineal en tiempo salvo la papa caliente, $\Theta(nk)$, y `quitarRepetidos` en sitio, $O(n^2)$ por el `eliminar` de la lista; el espacio es la estructura auxiliar que cada solución pide.

Lo que sigue

La STL de C++: los mismos tres contratos, ya implementados, con `vector`, `stack` y `queue`, y los primeros algoritmos sobre ellos.

Referencias I

- R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 7, pilas y sus aplicaciones; capítulo 8, colas.
- N. Kalicharan. *Data Structures in C*. 2008. Capítulo 4, pilas y colas.
- UVa Online Judge, problemas 673, 10935 y 514. `https://vjudge.net`